DOCUMENTAÇÃO

casalplanner.

Documentação Técnica e Funcional

Organizador de compras do casal



Versão 1.0 · Outubro de 2026
Autor do projeto: Matheus Rafael
github.com/MatheusRafaDev/casal-planner
casalplanner.vercel.app

Sumário

1. Visão geral 4
1.1 Propósito 4
1.2 Problema e público-alvo 4
1.3 Proposta de valor 4
1.4 Modos de conta 4
1.5 Informações do repositório 5

2. Funcionalidades 6
2.1 Dashboard (Início) 6
2.2 Planejamento por cômodos 6
2.3 Cadastro de itens (wizard de 3 etapas) 6
2.4 Busca e filtros 7
2.5 Pesquisa de preços online 7
2.6 Pesquisa por foto e histórico de preços 8
2.7 Scanner de código de barras 8
2.8 Inteligência artificial 8
2.9 Divisão de despesas e formas de pagamento 8
2.10 Parcelamento, juros e variantes 9
2.11 Origem do item e presentes 9
2.12 Lista de presentes pública 9
2.13 Dia da mudança e previsão do tempo 9
2.14 Conta, perfil e convites 10
2.15 PWA e notificações push 10
2.16 Exportação e compartilhamento 10

3. Arquitetura 11
3.1 Visão geral 11
3.2 Camadas do backend (Clean Architecture) 11
3.3 Ciclo de uma requisição 12
3.4 Decisões arquiteturais 12

4. Stack tecnológica 13
4.1 Frontend 13
4.2 Backend 13
4.3 Infraestrutura e serviços externos 14

5. Modelo de dados 15
5.1 Usuario 15
5.2 Categoria 16
5.3 Item 16
5.4 RegistroPrecoFoto 17

6. Referência da API 18
6.1 Autenticação — /api/auth 18
6.2 Usuário — /api/usuario 18
6.3 Recuperação de senha — /api/recuperarsenha 19
6.4 Cômodos — /api/categorias 19
6.5 Itens — /api/itens 19
6.6 Resumo, preços e IA 20
6.7 Lista pública e saúde 20

7. Autenticação e segurança 21
7.1 Sessão por cookies HttpOnly 21
7.2 Fluxos de autenticação 21
7.3 Limitação de requisições (rate limiting) 21
7.4 Demais medidas 22

8. Inteligência artificial e integrações 23
8.1 Cadeia de fallback da IA de texto 23
8.2 Pesquisa de preços e extração de links 23
8.3 Outras integrações 23

9. Frontend 25
9.1 Rotas 25

9.2 Estrutura de pastas 26
9.3 Camada de serviços 26
9.4 Estado, tema e responsividade 26
10. Design system e identidade visual 28
10.1 Logotipo 28
10.2 Paleta de cores 28
10.3 Cores do casal e dos cômodos 29
10.4 Tipografia, raios e demais tokens 29
10.5 Onde alterar a identidade visual 30
11. Configuração e execução 31
11.1 Pré-requisitos 31
11.2 Variáveis de ambiente do backend 31
11.3 Executando localmente 32
11.4 Scripts úteis 32
12. Deploy e CI/CD 33
12.1 Vercel 33
12.2 GitHub Actions 33
12.3 Manutenção do serviço gratuito 33
13. Testes e qualidade 34
14. Guia de uso 35
14.1 Primeiros passos 35
14.2 Cadastrando um item 35
14.3 Acompanhando as compras 35
14.4 Convidando o parceiro 35
14.5 Compartilhando uma lista de presentes 35
15. Observações de manutenção 36
15.1 Pontos técnicos para evolução 36
16. Glossário 37

1. Visão geral

1.1 Propósito

O **CasalPlanner** é uma aplicação web full-stack (instalável como PWA) para **planejar o enxoval e as compras da casa nova**. Ele transforma uma lista de compras grande e desorganizada em um planejamento estruturado por cômodo, com controle de orçamento, divisão de despesas entre duas pessoas, pesquisa de preços e recursos de inteligência artificial.

O slogan da marca é **"organizador de compras do casal"** e a identidade visual usa o verde-lar (#1F5C43) com o coral (#EF6F6C) como cor de acento.

1.2 Problema e público-alvo

Casais que estão montando a casa costumam controlar compras em planilhas, notas de celular e conversas de WhatsApp. Faltam uma visão consolidada do quanto já foi gasto, do que ainda falta comprar, de quem pagou cada item e de quanto do orçamento (em dinheiro e em vale-refeição/alimentação) já foi consumido. O CasalPlanner centraliza tudo isso em um único lugar, pensado para uso no celular.

O público principal são **casais em fase de mudança ou de montagem do lar**, mas o sistema também funciona para **uma única pessoa** (conta individual).

1.3 Proposta de valor

- **Organização por cômodo**: cada ambiente tem ícone, cor, orçamento próprio e barra de progresso.
- **Controle financeiro**: totais gastos e pendentes, meta global do enxoval, comparativo mensal e separação entre dinheiro e VR/VA.
- **Divisão entre duas pessoas**: cada item registra quem comprou e quanto cada um pagou.
- **Menos digitação**: autocomplete, sugestões por cômodo, detecção de duplicidade, pesquisa de preços online, leitura de código de barras e análise de fotos com IA.
- **Compartilhamento**: exportação em PDF, Web Share API e uma lista de presentes pública para amigos e familiares.
- **Experiência mobile-first**: PWA instalável com notificações push, navegação inferior no celular e sidebar no desktop.

1.4 Modos de conta

Aspecto	Conta individual	Conta casal
Cadastro	Uma pessoa, e-mail e senha (ou Google)	Duas pessoas cadastradas juntas (`registrar-casal`) ou vinculadas por convite
Divisão de despesas	Não se aplica	Valor de cada item dividido entre Pessoa 1 e Pessoa 2
Responsável pela compra	Não se aplica	Cada item registra quem comprou
Perfil	Nome, foto, meta global, tema	Perfil compartilhado com nome e avatar de cada pessoa

Aspecto	Conta individual	Conta casal
Convite	Pode convidar um parceiro por e-mail	Já formado; cada pessoa tem credenciais próprias
Notificações	Preferência do usuário	Preferência individual por pessoa

1.5 Informações do repositório

Item	Valor
Repositório	github.com/MatheusRafaDev/casal-planner (público, branch `main`)
Aplicação em produção	casalplanner.vercel.app
Autor	Matheus Rafael
Licença	MIT License (© 2026 Matheus Rafael)
Histórico	Cerca de 400 commits; documentação baseada no commit `b78436e` (26/10/2026)
Tamanho do código	Aproximadamente 10,6 mil linhas de C# (backend) e 16,4 mil linhas de TypeScript/TSX (frontend)
Estrutura	Monorepo: `backend/`, `frontend/`, `estilo/` (referências visuais), `.github/workflows/`

2. Funcionalidades

Este capítulo descreve o que o sistema faz do ponto de vista do usuário, relacionando cada recurso ao que existe no código.

2.1 Dashboard (Início)

A tela **Início** (`/inicio`) apresenta a visão geral do planejamento financeiro e do progresso do enxoval. Os dados vêm do endpoint `GET /api/resumo`, que devolve três blocos: o resumo atual, um comparativo mensal e o resumo do enxoval.

Bloco	Campos principais	O que mostra
Atual	`TotalGeral`, `TotalVR`, `TotalNormal`, `TotalComprados`, `TotalItens`, `TotalPessoa1`, `TotalPessoa2`, `PercentualConcluido`	Totais gerais, separação dinheiro × VR/VA, valor por pessoa e progresso
Por cômodo	`PorCategoria`, `QuantidadePorCategoria`, `CompradosPorCategoria`	Valor, quantidade e itens comprados em cada ambiente (alimenta os gráficos)
Comparativo	`MesAtual`, `MesPassado`, `MesRetrasado`, `PercentualGeral`	Gastos dos últimos três meses
Enxoval	`MetaGlobalEnxoval`, `PercentualMetaGlobal`, `TotalRestanteParaMeta`, `TotalItensComprados`, `TotalItensPendentes`	Quanto da meta global já foi atingido e quanto falta

- Alertas de orçamento excedido por cômodo.
- Gráficos de gastos (biblioteca Recharts) com a paleta oficial do design system.
- Widget de contagem regressiva para o **dia da mudança**, com previsão do tempo (ver 2.12).

2.2 Planejamento por cômodos

Os itens são agrupados em **categorias**, chamadas de cômodos na interface. Há cômodos padrão (por exemplo Sala, Cozinha, Quarto, Banheiro e Lavanderia) e o usuário pode criar os seus.

Campo	Descrição
`Nome`	Nome do cômodo
`Icon`	Ícone/emoji exibido (padrão 🗀); o frontend mantém um mapa de ícones em `icon-map.ts`
`Bg`	Cor de fundo/identidade do cômodo (padrão `#d6e9d6`)
`IsPadrao`	Indica se é um cômodo padrão do sistema
`Ordem`	Posição na lista; pode ser reordenada via `PUT /api/categorias/reordenar`
`MetaOrcamento`	Orçamento do cômodo (opcional); permite identificar cômodos que estouraram o limite

2.3 Cadastro de itens (wizard de 3 etapas)

O componente `AddItemWizard` conduz o usuário por um fluxo guiado:

1. **Identificação**: nome do produto com autocomplete, cômodo e quantidade.
2. **Pesquisa**: pesquisa de preços, comparação entre lojas, informações do produto, foto e parcelamento.
3. **Confirmação**: revisão dos dados, forma de pagamento, responsável pela compra e divisão do valor.

Também existe o `ItemFormModal` para edição rápida de qualquer item. As mutações usam **atualização otimista** (`onMutate` do TanStack Query), de modo que a interface responde imediatamente e reverte em caso de erro.

Campos de um item

Grupo	Campos
Identificação	`Nome`, `Marca`, `Loja`, `LinkProduto`, `FotoUrl` (+ `FotoPublicId` no Cloudinary)
Valores	`Preco`, `Quantidade`, `Parcelas`, `JurosMensal`
Classificação	`CategoriaId`, `Prioridade` (padrão `normal`), `Pagamento` (`normal` ou VR/VA), `Origem` e `OrigemDescricao`
Estado	`Comprado`, `DataCompra`, `CreatedAt`, `UpdatedAt`
Casal	`ResponsavelId` (1 ou 2) e `DivisaoPagamento` (`ValorPessoa1`, `ValorPessoa2`)
Variantes	`Variantes` e `VarianteSelecionadaId` (opções do mesmo produto com preço e loja diferentes)
Alertas e desfazer	`UltimoAlertaPrecoAt`, `UndoToken`, `UndoExpiresAt`

2.4 Busca e filtros

- Busca por nome e por marca.
- Filtros de status: todos, faltantes, comprados e recebidos como presente.
- Filtros de pagamento: dinheiro ou VR/VA.
- Filtro por responsável pela compra (Pessoa 1 / Pessoa 2).
- Listagem paginada no servidor (`GET /api/itens/page`) e listagem por cômodo (`GET /api/itens/categoria/{categoriaId}`).

2.5 Pesquisa de preços online

O `PainelPesquisaPrecos` consulta `GET /api/pesquisaprecos`. O `PesquisaPrecosService` dispara **todos os provedores registrados em paralelo**, cada um protegido por timeout e tratamento de falha individual — se um provedor cair, os demais continuam respondendo.

Provedor	Fonte	Observação
`GoogleShoppingProvider`	Google Shopping via SerpAPI	Requer `SERPAPI_KEY`
`MercadoLivreProvider`	API do Mercado Livre	Cliente HTTP dedicado (`MercadoLivreClient`)

O resultado é uma lista de `ProdutoDto` com nome, preço, preço original e desconto calculado, loja, indicador de marketplace, imagem, URL, frete, avaliação, disponibilidade, marca, modelo, fonte e um **score de confiança (0 a 100)**. O cache (Redis) usa 30 minutos por padrão (`PRICE_CACHE_MINUTES`) e o timeout padrão é de 20 segundos (`PRICE_TIMEOUT_SECONDS`).

Há ainda `POST /api/pesquisaprecos/extrair-link`, que lê uma URL de produto colada pelo usuário (serviço `ExtratorLinkService`, com apoio do `ScrapeDoService` e do HtmlAgilityPack) para preencher nome, preço e foto automaticamente.

2.6 Pesquisa por foto e histórico de preços

O recurso **Registro de preço por foto** (`PesquisaPrecosPorFoto` e `ConfirmarRegistroPreco`) permite fotografar um produto ou etiqueta no mercado. O fluxo tem três chamadas:

1. `POST /api/registropreco/analisar`: a imagem é analisada por modelos de visão (Gemini como principal e Groq Vision como alternativa). Limite de 50 chamadas por hora.
2. O usuário revisa e corrige os dados extraídos.
3. `POST /api/registropreco/confirmar`: grava o registro (produto, marca, preço, unidade, mercado, endereço com geocodificação e imagem) na coleção `RegistrosPreco`.

O histórico fica disponível em `GET /api/registropreco/historico`, servindo de referência para comparar preços pagos em diferentes mercados.

2.7 Scanner de código de barras

O `BarcodeScannerModal` usa a biblioteca **html5-qrcode** para ler códigos de barras pela câmera do celular. O arquivo `services/barcode.ts` converte o código lido em informações do produto para pré-preencher o cadastro.

2.8 Inteligência artificial

Recurso	Endpoint / componente	Descrição
Autocomplete	`lib/suggestions.ts` + wizard	Sugere produtos durante o cadastro
Sugestões por cômodo	`GET /api/groq/sugestoes-comodo`	Retorna itens que podem estar faltando (`Nome`, `Motivo`, `Estimativa`, `Prioridade`)
Detecção de duplicidade	`POST /api/groq/detectar-duplicata`	Informa se o item já existe (`Duplicata`, `ItemSimilar`, `Motivo`)
Estimativa de orçamento	`GET /api/groq/estimativa-comodo`	Faixas essencial, média e completa para montar um cômodo
Lojas e logos	`POST /api/groq/dominios`	Descobre o domínio/logo de cada loja a partir do nome
Identificação por imagem	`POST /api/registropreco/analisar`	Visão computacional sobre fotos de produtos

Os detalhes de provedores e fallback estão no capítulo 8.

2.9 Divisão de despesas e formas de pagamento

Em uma conta casal, cada item possui uma divisão personalizada de pagamento. O sistema oferece atalhos de **50/50**, **100/0** e **0/100** e valida automaticamente se a soma das partes é igual ao valor total.

```
Pessoa 1: R$ 500,00
Pessoa 2: R$ 500,00
-------------------
Total:    R$ 1.000,00
```

Cada compra é classificada como **dinheiro** (pagamento normal) ou **VR/VA**. O dashboard exibe os dois totais separadamente (`TotalNormal` e `TotalVR`).

2.10 Parcelamento, juros e variantes

- **Parcelamento**: `Parcelas` e `JurosMensal` permitem simular o valor de cada parcela (lógica em `lib/parcelas.ts`).
- **Variantes**: um mesmo item pode ter várias opções (`ItemVariante`: nome, preço, loja, foto, link e observação); a escolhida fica em `VarianteSelecionadaId`.
- **Prioridade**: classifica a urgência de cada item (padrão `normal`).

2.11 Origem do item e presentes

O campo `Origem` indica como o item entra no planejamento. O padrão é `comprado`; os demais valores tratados pelo código estão ligados a presentes (desejo, prometido e presente). Itens recebidos como presente não contam como despesa, de modo que o orçamento reflete apenas o que realmente sai do bolso do casal.

2.12 Lista de presentes pública

O casal pode ativar uma **lista de presentes pública**, acessível por um link com *slug* (`/lista/{slug}`), sem login. O controlador `ListaPublicaController` expõe três operações:

Operação	Rota	Comportamento
Ver lista	`GET /api/public/lista/{slug}`	Mostra o(s) nome(s) do casal e os itens elegíveis; retorna 404 se a lista não existir ou estiver inativa
Presentear	`POST /api/public/lista/{slug}/presentear/{itemId}`	O convidado informa o nome; a reserva é feita de forma atômica (retorna 409 se outra pessoa acabou de presentear) e o casal recebe um e-mail de aviso
Desfazer	`DELETE /api/public/lista/{slug}/presentear/{itemId}?token=…`	Cancela a reserva com o token recebido, dentro do prazo de validade

> **Proteções da lista pública**
> A rota é protegida por rate limiting: **5 requisições POST** e **60 GET** a cada 10 minutos por cliente. O token de desfazer expira (`UndoExpiresAt`) e só funciona para o item correspondente.

2.13 Dia da mudança e previsão do tempo

O usuário pode informar a **data da mudança** e a cidade (`PUT /api/usuario/data-mudanca`). O `WidgetDiaMudanca` mostra a contagem regressiva e, usando a API gratuita **Open-Meteo** (geocodificação e previsão de até 16 dias), exibe temperatura, chuva e vento esperados, além de um alerta de risco para o dia da mudança (`InfoRiscoMudanca`). A previsão é consultada diretamente pelo navegador (`services/weather.ts`).

2.14 Conta, perfil e convites

- Perfil individual: nome, foto (armazenada no Cloudinary), meta global do enxoval, tema e preferências de notificação.
- Perfil do casal: nome e avatar de cada pessoa (`PUT /api/usuario/perfil-casal/{id}`).
- Alteração de senha e exclusão de conta.
- **Convite de parceiro**: gera um token com validade e envia por e-mail; o convidado abre o link, cria ou acessa a conta e aceita o convite (`aceitar-convite`). `GET /api/usuario/meus-convites` lista convites pendentes.
- Tema claro, escuro ou do sistema, com preferência salva (`ModoEscuro`, padrão escuro no servidor).

2.15 PWA e notificações push

O frontend é um **PWA**: possui `manifest.json`, ícones (192, 512 e maskable) e um Service Worker (`sw.js`, cache `casal-planner-v7`) que pré-cacheia assets e usa estratégia cache-first para arquivos estáticos. O hook `usePwa` detecta se o app já está instalado e captura o evento `beforeinstallprompt` para oferecer a instalação.

As **notificações push** usam Web Push com chaves VAPID: o navegador se inscreve via `POST /api/usuario/push/subscribe`, a chave pública é obtida em `GET /api/usuario/push/vapidPublicKey`, e o backend envia as mensagens com a biblioteca WebPush (`PushService`). O usuário pode ativar ou desativar as notificações no perfil.

2.16 Exportação e compartilhamento

- **PDF**: relatório gerado no navegador com jsPDF + jsPDF-AutoTable, contendo itens, quantidades, valores, status e informações da compra.
- **Web Share API**: compartilha a lista em aplicativos como o WhatsApp, quando suportada.
- **Área de transferência**: cópia da lista como alternativa.

3. Arquitetura

3.1 Visão geral

O sistema é um **monorepo** com duas aplicações independentes: um frontend React (SPA/PWA) e uma API REST em ASP.NET Core. A API persiste dados no MongoDB, usa Redis como cache e integra serviços externos de IA, preços, e-mail e imagens.

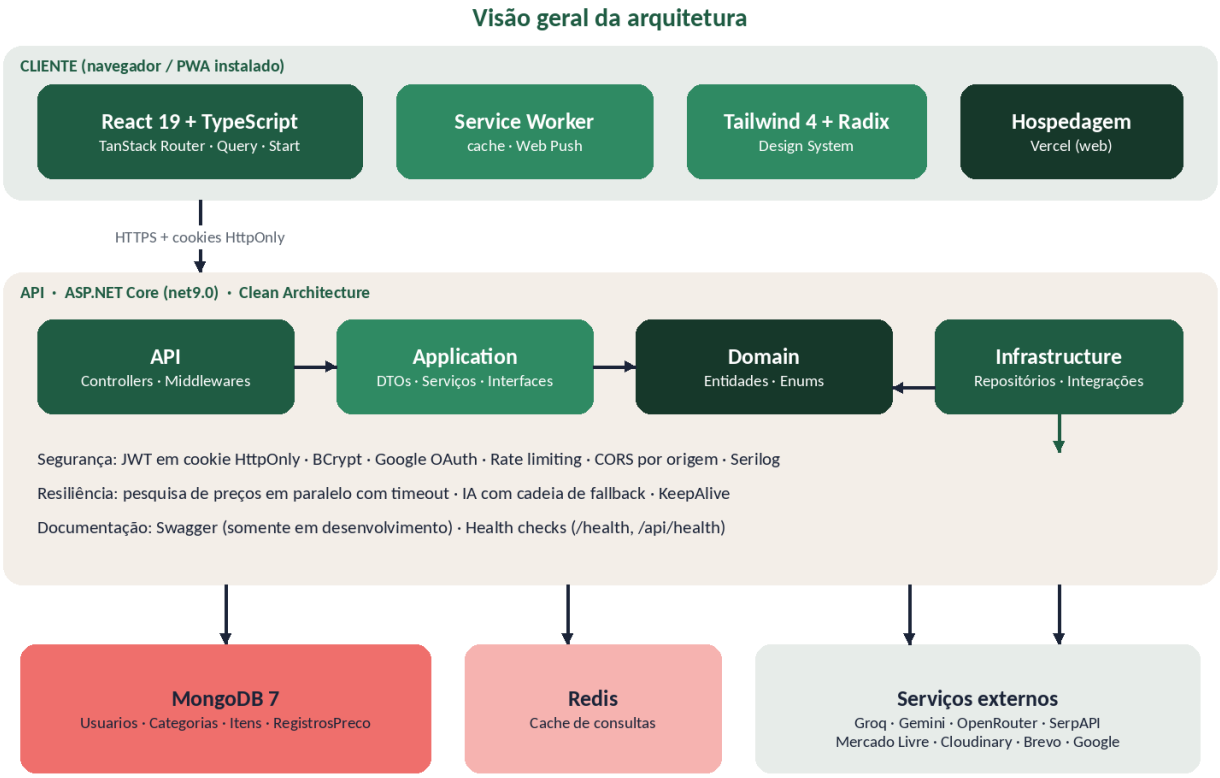


Figura 1 — Visão geral da arquitetura do CasalPlanner

3.2 Camadas do backend (Clean Architecture)

O backend segue a **regra de dependência da Clean Architecture**: as camadas externas dependem das internas, nunca o contrário.

```
API  ------>  Infrastructure  ------>  Application  ------>  Domain
API  ---------------------------------------->  Application

As setas apontam sempre para dentro: o Domain não depende de nenhuma outra camada.
```

Projeto	Responsabilidade	Conteúdo principal
`CasalPlanner.Domain`	Núcleo do negócio, sem dependências de framework	Entidades (`Usuario`, `CasalInfo`, `Item`, `Categoria`, `ItemVariante`, `RegistroPrecoFoto`), enums (`TipoConta`) e exceções

Projeto	Responsabilidade	Conteúdo principal
`CasalPlanner.Application`	Contratos e regras de orquestração	DTOs, interfaces (`IAuthService`, `IItemService`, `IItemRepository`…), `ItemService`, `ResumoService` e helpers de texto/preço
`CasalPlanner.Infrastructure`	Detalhes técnicos e integrações	`MongoDbContext`, repositórios, `AuthService`, `IaService`, serviços de visão, e-mail, push, cache Redis, Cloudinary, geocodificação, provedores de preço
`CasalPlanner.API`	Camada de entrada e composition root	Controllers, middlewares, `Program.cs` (DI, JWT, CORS, rate limit, Serilog, Swagger), `KeepAliveService`, Dockerfile
`CasalPlanner.Tests`	Testes unitários	xUnit + Moq

3.3 Ciclo de uma requisição

1. O navegador chama `/api/...` com `credentials: include`; o cookie de sessão `cp_token` acompanha a requisição.
2. O middleware de rate limiting (AspNetCoreRateLimit) verifica as cotas da rota e do cliente.
3. O pipeline autentica o JWT (lido do cookie) e verifica CORS/origem.
4. O Controller valida a entrada e chama o serviço de aplicação (por exemplo `IItemService`).
5. O serviço usa repositórios e serviços de infraestrutura (MongoDB, Redis, IA, e-mail…).
6. Exceções são tratadas pelo `ExceptionHandlingMiddleware`, que padroniza a resposta de erro; Serilog registra a requisição.

3.4 Decisões arquiteturais

Decisão	Motivação
MongoDB como banco principal	Esquema flexível para itens (variantes, divisão de pagamento, campos opcionais) e documentos de usuário que incorporam o `CasalInfo`
Autenticação por cookies HttpOnly	Tokens inacessíveis ao JavaScript, reduzindo o risco de roubo por XSS
Cadeia de fallback de IA	Usar provedores gratuitos com limites diferentes sem derrubar a aplicação quando um falha
Provedores de preço em paralelo	Menor latência e tolerância a falhas parciais
TanStack Router + Query	Rotas tipadas, cache de dados no cliente e atualizações otimistas
Design tokens centralizados	Troca de identidade visual em um único ponto (`design-system/`)
Frontend e API no mesmo domínio via Vercel	O rewrite `/api/*` evita problemas de CORS e de cookies de terceiros em produção

4. Stack tecnológica

4.1 Frontend

Tecnologia	Versão	Utilização
React	19.2	Interface
TypeScript	5.8	Tipagem estática
TanStack Router / Start	1.170 / 1.168	Roteamento baseado em arquivos, type-safe
TanStack Query	5.101	Cache e sincronização de dados do servidor
Tailwind CSS	4.2	Estilização com tokens de design
Radix UI + shadcn/ui	—	Componentes acessíveis (dialog, select, tabs, switch, checkbox, progress…)
Framer Motion	12	Animações (respeitando `prefers-reduced-motion`)
Recharts	2.15	Gráficos do dashboard
Zod	3.24	Validação de dados
jsPDF + AutoTable	4.2 / 5.0	Exportação em PDF
html5-qrcode	2.3	Leitura de código de barras
Sonner · Lucide · input-otp	—	Toasts, ícones e campo de código OTP
Vite	8	Build e servidor de desenvolvimento
Playwright	1.62	Testes end-to-end
ESLint + Prettier	9 / 3.7	Qualidade e formatação

4.2 Backend

Tecnologia	Versão	Utilização
ASP.NET Core / C#	.NET 9 (`net9.0`)	API REST
MongoDB.Driver	3.3	Acesso ao MongoDB
JwtBearer + System.IdentityModel.Tokens.Jwt	9.0 / 8.4	Autenticação por JWT
BCrypt.Net-Next	4.0	Hash de senhas
Google.Apis.Auth	1.75	Login com Google OAuth 2.0
WebPush	1.0.13	Notificações push (VAPID)
StackExchange.Redis	3.4	Cache
AspNetCoreRateLimit	5.0	Limitação de requisições
Serilog.AspNetCore	10.0	Logging estruturado

Tecnologia	Versão	Utilização
Swashbuckle (Swagger)	7.3	Documentação OpenAPI (apenas em desenvolvimento)
CloudinaryDotNet	1.29	Upload de imagens
MailKit · HtmlAgilityPack	4.17 / 1.12	E-mail e extração de HTML de links de produtos
Microsoft.Extensions.Http.Resilience / Polly	10.x	Resiliência em chamadas HTTP
xUnit + Moq	2.9 / 4.20	Testes unitários

4.3 Infraestrutura e serviços externos

Serviço	Utilização
MongoDB (Atlas ou equivalente)	Banco de dados principal (database padrão `CasalPlannerDB`)
Redis	Cache de consultas e de pesquisa de preços
Vercel	Hospedagem do frontend (serviço `web`) e da API (serviço `api`)
Docker + GitHub Container Registry	Imagem da API construída e publicada pelo GitHub Actions
Groq, Google Gemini, OpenRouter	IA de texto e visão
SerpAPI (Google Shopping) e Mercado Livre	Pesquisa de preços
Scrape.do	Extração de páginas de produtos
Cloudinary	Armazenamento de fotos de itens e avatares
Brevo	E-mails transacionais (recuperação de senha, convites, aviso de presente)
Open-Meteo e Nominatim	Previsão do tempo (frontend) e geocodificação de endereços (backend)
Google OAuth 2.0	Login social

5. Modelo de dados

O MongoDB possui quatro coleções, mapeadas pelas entidades do domínio por atributos `Bson*`. O identificador é um `ObjectId` representado como `string` no C#.

Coleção	Entidade	Descrição
`Usuarios`	`Usuario` (com `CasalInfo` embutido)	Contas individuais e de casal
`Categorias`	`Categoria`	Cômodos de cada usuário
`Itens`	`Item` (com `DivisaoPagamento`)	Itens do planejamento
`RegistrosPreco`	`RegistroPrecoFoto`	Histórico de preços registrados por foto

5.1 Usuario

Grupo	Campos	Observações
Identidade	`Id`, `Email`, `NomeCompleto`, `DataNascimento`, `Provider`	`Provider` é `local` ou Google
Conta	`TipoConta` (`Individual` | `Casal`), `IsCasal`, `IsAtivo`, `CreatedAt`, `LastLoginAt`	—
Credenciais	`SenhaHash`, `RefreshToken`, `RefreshTokenExpiraEm`	Senha com BCrypt
Recuperação de senha	`ResetCode`, `ResetCodeExpiresAt`, `ResetToken`, `ResetTokenExpiresAt`	Código por e-mail e token de redefinição, ambos com validade
Casal	`CasalInfo`	Documento embutido (ver abaixo)
Preferências	`ModoEscuro` (padrão `true`), `ReceberNotificacoes` (padrão `true`)	—
Meta e mudança	`MetaGlobalEnxoval`, `DataMudanca`, `CidadeMudanca`, `LatitudeMudanca`, `LongitudeMudanca`	Alimentam a meta global e o widget de previsão
Convite de parceiro	`ConviteParceiroToken`, `ConviteParceiroEmail`, `ConviteParceiroExpiraEm`	—
Lista pública	`SlugListaPublica`, `ListaPublicaAtiva`	Padrão desativada
Push	`PushSubscriptions` (lista de `PushSubscriptionInfo`)	Assinaturas Web Push dos dispositivos

CasalInfo (embutido em Usuario)

Repete, para **Pessoa 1** e **Pessoa 2**, os campos de identidade e credenciais: provedor, nome completo, e-mail, hash de senha, data de nascimento, avatar, preferência de notificações, campos de recuperação de senha e refresh token. Isso permite que as duas pessoas façam login de forma independente na mesma conta de casal.

5.2 Categoria

Campo	Tipo	Descrição
`Id`	ObjectId	Identificador

Campo	Tipo	Descrição
`Nome`	string	Nome do cômodo
`Icon`	string	Ícone/emoji (padrão 🗀)
`Bg`	string	Cor associada (padrão `#d6e9d6`)
`IsPadrao`	bool	Cômodo padrão do sistema
`Ordem`	int	Ordenação
`MetaOrcamento`	decimal?	Orçamento do cômodo
`CreatedAt` / `UpdatedAt`	DateTime	Auditoria
`UsuarioId`	ObjectId	Dono do cômodo

5.3 Item

Campo	Tipo	Descrição
`Id`, `UsuarioId`, `CategoriaId`	ObjectId	Identificadores (item, dono e cômodo)
`Nome`, `Marca`, `Loja`	string	Descrição do produto e onde comprar
`Preco`	decimal	Preço unitário
`Quantidade`	int	Padrão 1
`Comprado`	bool	Status de compra
`Pagamento`	string	`normal` (dinheiro) ou VR/VA; padrão `normal`
`Prioridade`	string	Padrão `normal`
`Origem`, `OrigemDescricao`	string	Como o item entrou no plano; padrão `comprado`
`LinkProduto`, `FotoUrl`, `FotoPublicId`	string	Link e imagem (Cloudinary)
`Parcelas`, `JurosMensal`	int, decimal	Parcelamento (padrão 1 parcela, 0% de juros)
`Variantes`, `VarianteSelecionadaId`	lista, string	Opções do item
`ResponsavelId`	int?	1 ou 2 — quem comprou
`DivisaoPagamento`	objeto	`ValorPessoa1` e `ValorPessoa2`
`DataCompra`	DateTime?	Quando foi comprado
`UltimoAlertaPrecoAt`	DateTime?	Controle de alertas de preço
`UndoToken`, `UndoExpiresAt`	string, DateTime?	Desfazer reserva de presente
`CreatedAt`, `UpdatedAt`	DateTime	Auditoria

5.4 RegistroPrecoFoto

Campo	Descrição
`Id`, `UsuarioId`, `ItemId`	Identificadores; `ItemId` é opcional
`ProdutoNome`, `Marca`, `Unidade`	Produto fotografado e unidade de medida
`Preco`	Preço registrado
`NomeMercado`, `Endereco`, `Latitude`, `Longitude`	Onde o preço foi visto, com coordenadas obtidas por geocodificação
`ImagemUrl`	Foto enviada
`DataCompra`, `CriadoEm`	Data do registro e de criação

6. Referência da API

Todas as rotas começam com `/api`. As rotas marcadas como **Sessão** exigem o cookie `cp_token` válido (ver capítulo 7); as marcadas como **Pública** não exigem autenticação. Em desenvolvimento, o Swagger fica disponível em `/swagger`.

6.1 Autenticação — /api/auth

Método	Rota	Acesso	Descrição
POST	`/api/auth/login`	Pública	Login com e-mail e senha; grava os cookies de sessão
POST	`/api/auth/google`	Pública	Login/cadastro com Google (ID token)
POST	`/api/auth/refresh`	Cookie de refresh	Renova a sessão usando `cp_refresh_token`
POST	`/api/auth/logout`	Sessão	Encerra a sessão e remove os cookies
GET	`/api/auth/me`	Sessão	Dados do usuário autenticado

6.2 Usuário — /api/usuario

Método	Rota	Acesso	Descrição
POST	`/api/usuario/registrar`	Pública	Cadastro individual
POST	`/api/usuario/registrar-casal`	Pública	Cadastro de casal
GET	`/api/usuario/me`	Sessão	Perfil completo do usuário
PUT	`/api/usuario/perfil`	Sessão	Atualiza o perfil individual
PUT	`/api/usuario/perfil-casal/{id}`	Sessão	Atualiza o perfil do casal
PUT	`/api/usuario/modo-escuro/{id}`	Sessão	Altera a preferência de tema
POST	`/api/usuario/alterar-senha`	Sessão	Altera a senha
PUT	`/api/usuario/lista-publica`	Sessão	Ativa/desativa a lista de presentes pública
PUT	`/api/usuario/data-mudanca`	Sessão	Define data e cidade da mudança
PUT	`/api/usuario/notificacoes`	Sessão	Liga/desliga notificações
DELETE	`/api/usuario/usuario/{id}`	Sessão	Exclui a conta
POST	`/api/usuario/convite`	Sessão	Convida o parceiro por e-mail
GET	`/api/usuario/convite/{token}`	Pública	Consulta um convite pelo token
GET	`/api/usuario/meus-convites`	Sessão	Lista convites do usuário
POST	`/api/usuario/aceitar-convite`	Sessão	Aceita um convite
GET	`/api/usuario/push/vapidPublicKey`	Pública	Chave pública VAPID
POST	`/api/usuario/push/subscribe`	Sessão	Registra a assinatura de push do dispositivo

6.3 Recuperação de senha — /api/recuperarsenha

Método	Rota	Acesso	Descrição
POST	`/api/recuperarsenha/esqueci-senha`	Pública	Envia um código de recuperação por e-mail
POST	`/api/recuperarsenha/validar-codigo`	Pública	Valida o código e libera a redefinição
POST	`/api/recuperarsenha/redefinir-senha`	Pública	Define a nova senha

6.4 Cômodos — /api/categorias

Método	Rota	Acesso	Descrição
GET	`/api/categorias`	Sessão	Lista os cômodos
GET	`/api/categorias/{id}`	Sessão	Detalha um cômodo
GET	`/api/categorias/usuario`	Sessão	Lista os cômodos do usuário logado
POST	`/api/categorias`	Sessão	Cria um cômodo
PUT	`/api/categorias/{id}`	Sessão	Edita um cômodo
PUT	`/api/categorias/reordenar`	Sessão	Reordena os cômodos
DELETE	`/api/categorias/{id}`	Sessão	Exclui um cômodo

6.5 Itens — /api/itens

Método	Rota	Acesso	Descrição
GET	`/api/itens`	Sessão	Lista todos os itens
GET	`/api/itens/page`	Sessão	Listagem paginada com filtros
GET	`/api/itens/{id}`	Sessão	Detalha um item
GET	`/api/itens/categoria/{categoriaId}`	Sessão	Itens de um cômodo
POST	`/api/itens`	Sessão	Cria um item
PUT	`/api/itens/{id}`	Sessão	Edita um item
PATCH	`/api/itens/{id}/comprado`	Sessão	Marca/desmarca como comprado
PUT	`/api/itens/{id}/categoria`	Sessão	Move o item para outro cômodo
DELETE	`/api/itens/{id}`	Sessão	Exclui um item

6.6 Resumo, preços e IA

Método	Rota	Acesso	Descrição
GET	`/api/resumo`	Sessão	Totais, comparativo mensal e progresso da meta

Método	Rota	Acesso	Descrição
GET	`/api/pesquisaprecos`	Sessão	Pesquisa de preços em múltiplos provedores
POST	`/api/pesquisaprecos/extrair-link`	Sessão	Extrai dados de um link de produto
POST	`/api/registropreco/analisar`	Sessão	Analisa uma foto de produto/etiqueta (50/hora)
POST	`/api/registropreco/confirmar`	Sessão	Confirma e grava o registro de preço
GET	`/api/registropreco/historico`	Sessão	Histórico de preços registrados
GET	`/api/groq/sugestoes-comodo`	Sessão	Sugestões de itens por cômodo
POST	`/api/groq/detectar-duplicata`	Sessão	Detecta itens duplicados
GET	`/api/groq/estimativa-comodo`	Sessão	Estimativa de orçamento por cômodo
POST	`/api/groq/dominios`	Sessão	Domínio e logo de lojas

O controlador de IA responde também sob o prefixo alternativo `/api/ia`.

6.7 Lista pública e saúde

Método	Rota	Acesso	Descrição
GET	`/api/public/lista/{slug}`	Pública	Lista de presentes pública
POST	`/api/public/lista/{slug}/presentear/{itemId}`	Pública	Reserva um presente
DELETE	`/api/public/lista/{slug}/presentear/{itemId}`	Pública	Desfaz a reserva (token)
GET	`/health`	Pública	Estado da API, ambiente e origens permitidas
GET	`/api/health`	Pública	Resposta simples `{ status: "Healthy", timestamp }`
GET	`/`	Pública	Mensagem de confirmação de que a API está no ar

> **Convenções**
> Os nomes de rotas, campos e mensagens estão em português. Erros de validação e de negócio retornam JSON com a propriedade `message`. Códigos usuais: 400 (dados inválidos), 401 (sem sessão), 404 (não encontrado), 409 (conflito, como no presente já reservado) e 429 (limite de requisições excedido).

7. Autenticação e segurança

7.1 Sessão por cookies HttpOnly

A API autentica as sessões por **cookies HttpOnly**. Os tokens de acesso e de renovação **não** são devolvidos nos JSONs, **não** são armazenados pelo frontend e **não** são aceitos pelo cabeçalho `Authorization`. Clientes web precisam enviar as requisições com credenciais incluídas (`credentials: "include"`).

Cookie	Conteúdo	Validade	Atributos
`cp_token`	JWT de acesso	`JWT_EXPIRES_IN` minutos	HttpOnly; Secure (exceto em dev); SameSite=None em produção e Lax em dev
`cp_refresh_token`	Token de renovação	7 dias (clientes mobile: 3650 dias)	Mesmos atributos

No frontend, o módulo `lib/api.ts` envia todas as chamadas com credenciais; ao receber **401**, tenta **uma única renovação** em `/api/auth/refresh` (compartilhada entre chamadas simultâneas) e repete a requisição original. Se a renovação falhar, o usuário é levado ao login.

7.2 Fluxos de autenticação

Fluxo	Passos
Login local	E-mail e senha → verificação do hash BCrypt → emissão do JWT e do refresh → cookies → atualização de `LastLoginAt`
Google OAuth 2.0	O frontend obtém o ID token do Google → `POST /api/auth/google` → validação com `Google.Apis.Auth` (`GOOGLE_CLIENT_ID`) → cria ou localiza o usuário
Recuperação de senha	`esqueci-senha` (código por e-mail via Brevo) → `validar-codigo` → `redefinir-senha`; código e token têm prazo de validade
Convite de parceiro	Token único com expiração enviado por e-mail → `aceitar-convite` vincula as contas

7.3 Limitação de requisições (rate limiting)

O AspNetCoreRateLimit está configurado com regras por rota. Em desenvolvimento, o limite de autenticação é elevado para 100.

Rota	Limite	Janela
Qualquer rota (*)	100 requisições	1 minuto
`POST /api/auth/*`	10 (100 em dev)	10 minutos
`POST /api/recuperarsenha/*`	10 (100 em dev)	10 minutos
`POST /api/registropreco/analisar`	50	1 hora
`POST /api/public/lista/*`	5	10 minutos
`GET /api/public/lista/*`	60	10 minutos

7.4 Demais medidas

- **BCrypt** para armazenar senhas; senhas nunca são retornadas pela API.
- **CORS** configurável por `CORS_ALLOWED_ORIGINS`; requisições com cookie de sessão têm a origem verificada contra a lista permitida.
- **Validação** de entrada no frontend com Zod e no backend nos controllers/serviços (por exemplo, soma da divisão de pagamento).
- **Tokens temporários** com expiração para recuperação de senha, convites e desfazer de presentes.
- **Segredos em variáveis de ambiente** (`.env` fora do Git); `JWT_SECRET_KEY` exige no mínimo 32 caracteres.
- **Logs estruturados** com Serilog e tratamento centralizado de exceções.
- **Swagger apenas em desenvolvimento**, evitando expor a documentação em produção.

> **Ponto de atenção**
> O rate limiting usa armazenamento **em memória** (`AddInMemoryRateLimiting`). Em um cenário com várias instâncias da API, cada instância mantém contadores próprios; para limites globais seria necessário um armazenamento compartilhado (por exemplo, o Redis que o projeto já utiliza).

8. Inteligência artificial e integrações

8.1 Cadeia de fallback da IA de texto

O `IaService` tenta os provedores em ordem e usa o primeiro que responder. Cada provedor só é considerado se a chave de API correspondente estiver configurada.

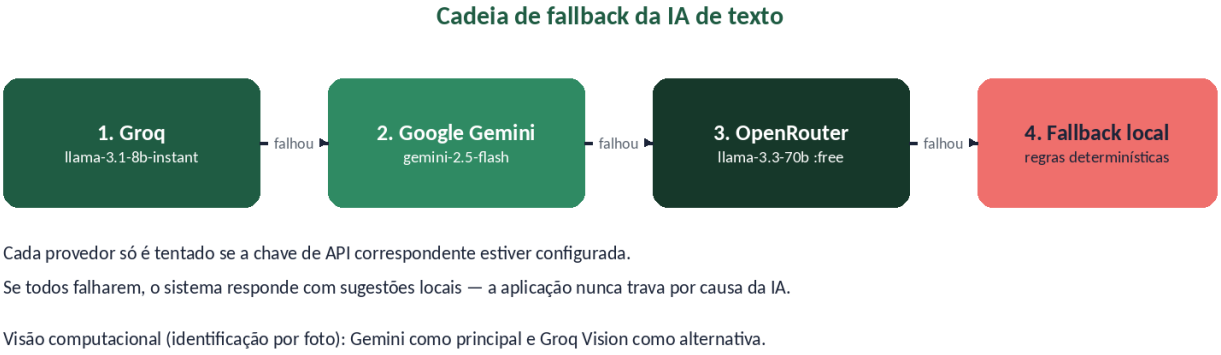


Figura 2 — Ordem de tentativa dos provedores de IA de texto

Ordem	Provedor	Modelo	Variável
1	Groq	`llama-3.1-8b-instant`	`GROQ_API_KEY`
2	Google Gemini	`gemini-2.5-flash`	`GEMINI_API_KEY`
3	OpenRouter	`meta-llama/llama-3.3-70b-instruct:free`	`OPENROUTER_API_KEY`
4	Fallback local	Regras determinísticas	—

A variável `IA_CALL_DELAY_MS` adiciona um atraso artificial opcional entre a requisição e a chamada à IA (0 desativa). Para **visão computacional** (fotos), o Gemini é o provedor principal e o Groq Vision é a alternativa (`GeminiVisionService`, `GroqVisionService` e `VisionAnalysisService`).

8.2 Pesquisa de preços e extração de links

- `PesquisaPrecosService` executa os provedores `GoogleShoppingProvider` e `MercadoLivreProvider` em paralelo, com timeout e isolamento de falhas.
- `PriceTextHelper` e `StoreAndBrandHelper` normalizam textos de preço, lojas e marcas.
- `ExtratorLinkService` e `ScrapeDoService` leem páginas de produto a partir de uma URL.
- `RedisCacheService` armazena resultados para reduzir chamadas pagas.

8.3 Outras integrações

Integração	Classe	Uso
Brevo	`EmailService`	Recuperação de senha, convite de parceiro e aviso de presente recebido
Cloudinary	`CloudinaryService`	Upload de fotos de itens e avatares

Integração	Classe	Uso
Web Push (VAPID)	`PushService`	Envio de notificações aos dispositivos inscritos
Nominatim	`GeocodingService`	Sugestão e geocodificação de endereços para o registro de preços
Open-Meteo	`services/weather.ts` (frontend)	Busca de cidades e previsão para o dia da mudança
Google OAuth	`AuthService` / `AuthController`	Login social
KeepAlive	`KeepAliveService`	Chama `KEEP_ALIVE_URL` a cada 14 minutos para evitar que a API hospedada em plano gratuito (Render) adormeça

9. Frontend

9.1 Rotas

O roteamento usa o **TanStack Router** baseado em arquivos (`src/routes`). As rotas sob `_authenticated` exigem sessão ativa; o arquivo `route.tsx` desse grupo redireciona para o login quando não há usuário.

Rota	Arquivo	Acesso	Descrição
`/`	`routes/index.tsx`	Pública	Página inicial de apresentação do produto
`/login`	`routes/login.tsx`	Pública	Login, cadastro individual/casal e entrada com Google
`/recuperar-senha`	`routes/recuperar-senha.tsx`	Pública	Assistente de recuperação de senha (e-mail → código → nova senha)
`/trocar-senha`	`routes/trocar-senha.tsx`	Pública (fora de `_authenticated`)	Troca de senha reutilizando o assistente de recuperação; página `noindex`
`/lista/$slug`	`routes/lista/$slug.tsx`	Pública	Lista de presentes compartilhada
`/inicio`	`_authenticated/inicio.tsx`	Sessão	Dashboard
`/planejamento`	`_authenticated/planejamento.tsx`	Sessão	Cômodos, itens, filtros e cadastro
`/perfil`	`_authenticated/perfil.tsx`	Sessão	Perfil, casal, convites, tema, notificações e lista pública

9.2 Estrutura de pastas

```
frontend/
├── public/            manifest.json, sw.js, ícones, logos, ilustrações
├── e2e/               testes Playwright
└── src/
    ├── routes/            páginas (TanStack Router, file-based)
    ├── components/
    │   ├── auth/          LoginForm, RegistroIndividual, RegistroCasal, WizardRecuperarSenha
    │   ├── layout/        AppShell (sidebar no desktop, navegação inferior no mobile)
    │   ├── planejamento/  AddItemWizard, ItemFormModal, CategoriaFormModal,
    │   │                  PainelPesquisaPrecos, PesquisaPrecosPorFoto,
    │   │                  ConfirmarRegistroPreco, BarcodeScannerModal, WidgetDiaMudanca
    │   └── ui/            componentes base (shadcn/Radix), BrandLogo, LogoBadge
    ├── services/          camada de acesso à API (um arquivo por domínio)
    ├── lib/               api.ts, auth-context, theme-context, formatters, parcelas,
suggestions
    ├── hooks/             use-pwa, use-mobile, use-reduced-motion
    └── design-system/     brand.ts, tokens.ts, index.ts
```

9.3 Camada de serviços

Arquivo	Responsabilidade
`services/auth.ts`	Login, Google, logout, refresh e usuário atual
`services/usuario.ts`	Cadastro, perfil, senha, notificações, lista pública e data da mudança
`services/categorias.ts` · `itens.ts` · `resumo.ts`	CRUD de cômodos e itens e dados do dashboard
`services/pesquisa-precos.ts` · `registro-preco.ts`	Pesquisa de preços, extração de link e registro por foto
`services/groq.ts`	Sugestões, duplicidade, estimativas e domínios de lojas
`services/convite.ts` · `recuperar-senha.ts`	Convites e recuperação de senha
`services/lista-publica.ts` · `weather.ts` · `barcode.ts`	Lista de presentes, previsão do tempo e código de barras
`services/types.ts`	Tipos TypeScript compartilhados

9.4 Estado, tema e responsividade

- **Dados do servidor**: TanStack Query, com invalidação após mutações e atualizações otimistas nos formulários.
- **Autenticação**: `auth-context.tsx` mantém o usuário atual e expõe login/logout.
- **Tema**: `theme-context.tsx` suporta claro, escuro e sistema, guardando a escolha em `localStorage` e aplicando a classe `.dark`.
- **Layout adaptativo**: `AppShell` usa navegação inferior no celular e sidebar/header em telas maiores (`use-mobile`).
- **Acessibilidade e conforto**: componentes Radix e respeito a `prefers-reduced-motion`.
- **Erros**: `error-capture.ts` e `error-page.ts` centralizam a captura e a exibição de falhas inesperadas.

10. Design system e identidade visual

O CasalPlanner usa **design tokens** centralizados. Alterar o nome da marca, o logotipo, as cores ou a paleta de cômodos exige editar apenas alguns arquivos; todos os componentes se atualizam automaticamente.

10.1 Logotipo



casalplanner.

O ícone é um quadrado arredondado em **verde-lar** com dois cartões sobrepostos: um coral ao fundo e um branco à frente, que traz a letra **"C."**, um coração e uma lista de verificação — simbolizando a lista de compras feita a dois. O logotipo horizontal escreve **"casal"** em peso 800 (verde), **"planner"** em peso 300 (tinta) e um ponto final em coral.

Arquivo (frontend/public)	Uso
`logo.svg`	Logotipo horizontal (ícone + texto), 280×56
`logo-icon.svg`	Apenas o ícone, 160×160
`logo-texto.svg` · `logo-texto-dark.svg`	Apenas o texto, para fundo claro e escuro
`logo.png` · `logo_casalplanner.png`	Ícone em PNG 512×512
`favicon.svg` · `favicon.png` · `favicon.ico`	Favicons
`apple-touch-icon.png`	Ícone para iOS
`icons/icon-192.png` · `icon-512.png` · `icon-512-maskable.png`	Ícones do PWA

10.2 Paleta de cores

Cor	Nome	Hex	Uso no sistema
	Verde Lar	`#1F5C43`	Cor primária: títulos, botões, fundos de marca; Pessoa 1; cor do tema do PWA
	Verde Folha	`#2F8A63`	Hover, sucesso, despesas conjuntas e primária no modo escuro
	Verde Floresta	`#16382A`	Fundo do modo escuro e contraste
	Coral	`#EF6F6C`	Acento: ponto da marca, coração, destaques, alertas; Pessoa 2
	Coral Claro	`#F6B3B0`	Fundos suaves de acento e avisos

Cor	Nome	Hex	Uso no sistema
	Creme	#FAF6F1	Fundo geral das telas (modo claro)
	Tinta	#1B2438	Texto principal
	Névoa	#E7ECE9	Divisores e fundos suaves
	Cinza	#5B6470	Texto secundário
	Borda	#EEE3D8	Bordas de cartões (2 px)
	Azul Petróleo	#2A7F8E	Informação e cômodo Banheiro
	Roxo	#8B5CF6	Presente / ganho
	Destrutivo	#DC2626	Erros e ações de exclusão

10.3 Cores do casal e dos cômodos

Elemento	Cor	Observação
Pessoa 1	Verde Lar #1F5C43	No modo escuro usa #2F8A63
Pessoa 2	Coral #EF6F6C	Complementar ao verde
Conjunto	Verde Folha #2F8A63	No modo escuro usa #38B2AC
Presente	Roxo #8B5CF6	No modo escuro usa #A78BFA

Cômodo	Cor	Cômodo	Cor
Sala	#1F5C43	Escritório	#5B6470
Cozinha	#EF6F6C	Varanda	#C85A32
Quarto	#2F8A63	Garagem	#344C42
Banheiro	#2A7F8E	Jardim	#5E6E3B
Lavanderia	#16382A	Gourmet	#BC8A2C
Closet	#7A3E4D	—	—

A sequência de cores dos gráficos do dashboard é: #1F5C43, #EF6F6C, #2F8A63, #16382A, #F6B3B0, #5B6470 e #2A7F8E.

10.4 Tipografia, raios e demais tokens

Token	Valor
Fonte principal, de títulos e da marca	Plus Jakarta Sans (fallback: system-ui)
Fonte manuscrita (detalhes)	Caveat
Fonte monoespaçada	ui-monospace, SFMono-Regular, Menlo, Consolas
Raios de borda	sm 8 px · md 10 px · lg 12 px · xl 16 px · full 9999 px

Token	Valor
Tema do PWA (`theme_color`)	`#1F5C43`; fundo (`background_color`) `#FAF6F1`

10.5 Onde alterar a identidade visual

Arquivo	Finalidade
`frontend/src/design-system/brand.ts`	Nome, slogan, caminhos dos logos e cores do PWA (`BRAND_CONFIG`)
`frontend/src/design-system/tokens.ts`	Paletas (primária, acento, casal, cômodos, gráficos), tipografia, raios e sombras
`frontend/src/design-system/index.ts`	Exportação unificada e utilitários (por exemplo `getRoomDefaultColor`)
`frontend/src/styles.css`	Variáveis CSS (`:root`, `.dark`, `@theme`) sincronizadas com o Tailwind 4
`frontend/src/components/ui/BrandLogo.tsx`	Componente oficial do logotipo (ícone + texto)

Classes utilitárias semânticas disponíveis no Tailwind: `bg-primary`, `text-primary`, `bg-p1`, `bg-p2`, `bg-joint`, `bg-card`, `border-border`, `bg-brand-dot` e a escala `rounded-lg/xl/2xl`. A pasta `estilo/` do repositório guarda as referências do redesign (PDFs das telas Início, Planejamento e Visitante, e o prompt de redesign).

11. Configuração e execução

11.1 Pré-requisitos

- **.NET SDK 9**
- **Node.js** (versão compatível com Vite 8) e npm
- Uma instância de **MongoDB** (local ou Atlas)
- Opcional: Redis, Docker e chaves das integrações (IA, SerpAPI, Cloudinary, Brevo)

11.2 Variáveis de ambiente do backend

Copie `backend/CasalPlanner.API/.env.example` para `.env` e preencha os valores. Nunca versione o `.env`. Para rodar localmente, **apenas `JWT_SECRET_KEY` e `MONGODB_CONNECTION_STRING` são indispensáveis**; as demais habilitam recursos específicos.

Variável	Obrigatória	Descrição
`JWT_SECRET_KEY`	Sim	Segredo de assinatura do JWT (mínimo 32 caracteres)
`MONGODB_CONNECTION_STRING`	Sim	String de conexão do MongoDB
`MONGODB_DATABASE`	Não	Nome do banco (padrão `CasalPlannerDB`)
`JWT_ISSUER` · `JWT_AUDIENCE`	Não	Padrões `CasalPlanner` e `CasalPlannerUsers`
`JWT_EXPIRES_IN`	Não	Validade do JWT em minutos
`CORS_ALLOWED_ORIGINS`	Produção	Origens permitidas, separadas por vírgula
`FRONTEND_URL`	Recomendada	URL do frontend (links de e-mail e convites)
`GOOGLE_CLIENT_ID`	Para login Google	Client ID do Google OAuth
`GROQ_API_KEY`	Para IA	IA de texto (principal) e visão (alternativa)
`GEMINI_API_KEY`	Para IA	IA de visão (principal) e texto (fallback)
`OPENROUTER_API_KEY`	Opcional	Fallback adicional com modelos `:free`
`IA_CALL_DELAY_MS`	Opcional	Atraso artificial antes das chamadas de IA
`SERPAPI_KEY`	Para preços	Google Shopping via SerpAPI
`SCRAPEDO_TOKEN`	Opcional	Extração de páginas de produto
`PRICE_CACHE_MINUTES` · `PRICE_TIMEOUT_SECONDS`	Não	Padrões 30 e 20
`REDIS_CONNECTION_STRING` / `REDIS_URL`	Opcional	Conexão do Redis
`CLOUDINARY_CLOUD_NAME` · `_API_KEY` · `_API_SECRET`	Para fotos	Upload de imagens
`BREVO_API_KEY` · `BREVO_FROM_EMAIL`	Para e-mails	Sem elas, recuperação de senha e notificações por e-mail não funcionam

Variável	Obrigatória	Descrição
`VAPID_SUBJECT` · `VAPID_PUBLIC_KEY` · `VAPID_PRIVATE_KEY`	Para push	Chaves Web Push
`KEEP_ALIVE_URL`	Opcional	URL do health check para manter a API acordada em plano gratuito
`PORT` · `ASPNETCORE_ENVIRONMENT`	Não	Porta e ambiente

No frontend, a única variável relevante é `VITE_API_BASE_URL`; em desenvolvimento local ela deve ficar **vazia**, pois o Vite encaminha `/api` para a API local.

11.3 Executando localmente

1. Configure o `.env` do backend (ver acima).
2. Inicie a API: `cd backend/CasalPlanner.API` e `dotnet run --launch-profile http` (porta `5286`).
3. Em outro terminal: `cd frontend`, `npm install` e `npm run dev`.
4. Acesse `http://localhost:8080`. O servidor de desenvolvimento encaminha `/api` para `http://localhost:5286`.

```
# API
cd backend/CasalPlanner.API
dotnet run --launch-profile http

# Frontend
cd frontend
npm install
npm run dev
```

11.4 Scripts úteis

Comando	Efeito
`npm run build` (raiz ou `frontend/`)	Build de produção do frontend
`npm run lint` · `npm run format`	ESLint e Prettier
`npm run test:e2e`	Testes Playwright
`dotnet test` (pasta `backend`)	Testes xUnit
`docker build -f backend/CasalPlanner.API/Dockerfile ./backend`	Imagem Docker da API (build multi-stage)

12. Deploy e CI/CD

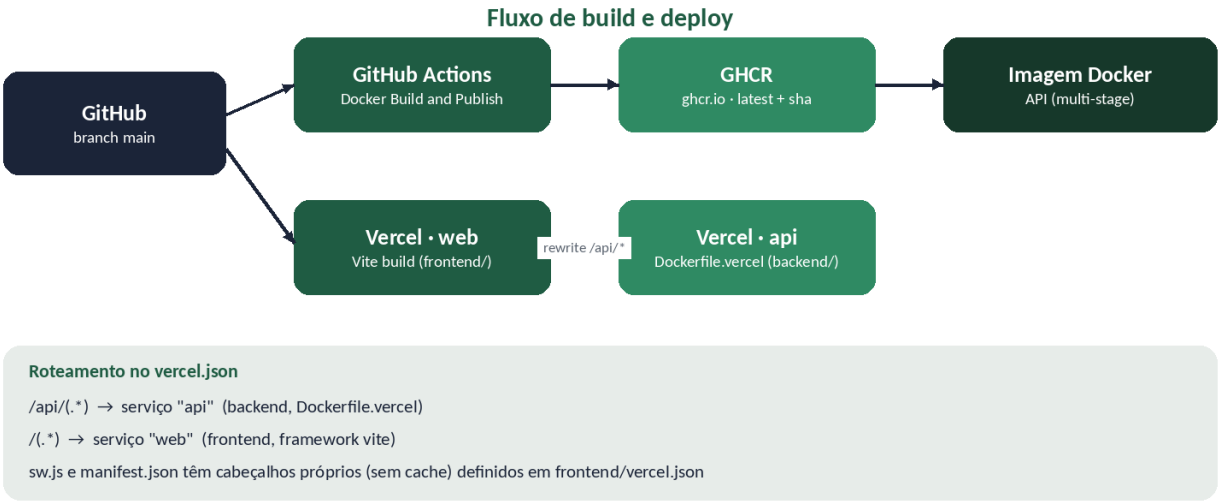


Figura 3 — Fluxo de build e deploy

12.1 Vercel

O arquivo `vercel.json` da raiz define dois **serviços**: `web` (pasta `frontend`, framework Vite, `npm run build`) e `api` (pasta `backend`, entrypoint `Dockerfile.vercel`). Os rewrites encaminham `/api/(.*)` para o serviço `api` e todo o resto para `web`, de modo que navegador e API compartilham o mesmo domínio.

O `frontend/vercel.json` define cabeçalhos específicos: `sw.js` sem cache e com `Service-Worker-Allowed: /`, `manifest.json` sem cache, ícones com cache imutável de um ano, e `robots.txt` e `sitemap.xml` com cache de um dia.

12.2 GitHub Actions

O workflow **Docker Build and Publish** executa a cada push na branch `main`: faz checkout, autentica no GitHub Container Registry (`ghcr.io`), gera as tags `latest` e o SHA curto do commit, e constrói e publica a imagem usando `backend/CasalPlanner.API/Dockerfile` com contexto `./backend`.

12.3 Manutenção do serviço gratuito

Para planos gratuitos que adormecem após inatividade, o `KeepAliveService` envia uma requisição a `KEEP_ALIVE_URL` a cada 14 minutos. Sem essa variável, o serviço se desativa e registra um aviso no log.

13. Testes e qualidade

Camada	Ferramenta	Cobertura
Backend	xUnit + Moq	`AuthServiceTests`, `ItemServiceTests`, `IaServiceTests` e `PriceTextHelperTests` — autenticação, regras de itens, IA com fallback e tratamento de texto/preço
Frontend	Playwright	Testes end-to-end em `frontend/e2e` (por exemplo, rolagem em dispositivos móveis)
Qualidade estática	ESLint 9 + Prettier + TypeScript	Configurados em `eslint.config.js` e `tsconfig.json`

Há espaço para ampliar a cobertura, principalmente nos controllers, no `ResumoService` e na lista pública, que concentram regras de negócio relevantes.

14. Guia de uso

14.1 Primeiros passos

1. Acesse casalplanner.vercel.app e escolha **criar conta** como pessoa individual ou como casal (ou entre com o Google).
2. No perfil, defina a **meta global do enxoval** e, se quiser, a **data da mudança**.
3. Em **Planejamento**, revise os cômodos padrão ou crie os seus, escolhendo ícone, cor e orçamento.
4. Instale o app no celular pelo navegador (opção “Adicionar à tela inicial”/instalar) e ative as notificações.

14.2 Cadastrando um item

1. Toque em **adicionar item** e digite o nome; use as sugestões do autocomplete ou escaneie o código de barras.
2. Escolha o cômodo e a quantidade e avance para a pesquisa de preços; selecione a oferta desejada ou informe o preço manualmente.
3. Confirme a forma de pagamento (dinheiro ou VR/VA), quem comprou e como o valor foi dividido.
4. Salve. O item aparece no cômodo e o dashboard é atualizado.

14.3 Acompanhando as compras

- Marque o item como **comprado** quando efetivar a compra.
- Use os filtros para ver o que falta, o que já foi comprado e o que veio de presente.
- Consulte o dashboard para ver progresso da meta, gastos por cômodo e totais em dinheiro × VR/VA.
- Exporte a lista em PDF ou compartilhe pelo WhatsApp.

14.4 Convidando o parceiro

1. No perfil, informe o e-mail da pessoa e envie o convite.
2. A pessoa recebe um e-mail com um link e prazo de validade.
3. Ao aceitar, as contas são vinculadas e passam a compartilhar o planejamento.

14.5 Compartilhando uma lista de presentes

1. No perfil, ative a **lista pública** e copie o link (`/lista/seu-slug`).
2. Envie o link aos convidados. Eles escolhem um item e informam o nome.
3. Você recebe um e-mail a cada presente prometido; o convidado pode desfazer a reserva dentro do prazo.

15. Observações de manutenção

Ao comparar o README com o código na data desta documentação, encontrei diferenças que vale corrigir para evitar confusão em novos colaboradores:

Tema	README / documentação diz	Código atual
Versão do .NET	ASP.NET Core 8 / .NET 8	Projetos com `net9.0`
Biblioteca de gráficos	ApexCharts e Recharts	`package.json` contém apenas Recharts
Provedores de preço	SerpAPI, Mercado Livre e Amazon	Registrados: Google Shopping (SerpAPI) e Mercado Livre
E-mail	Resend / SMTP	Brevo (`EmailService`); MailKit como dependência
Autenticação	"JWT Bearer"	Sessão por cookies HttpOnly; o cabeçalho `Authorization` não é aceito
Gerador de chaves VAPID	`dotnet run --project backend/KeyGen`	Pasta `KeyGen` não existe no repositório
Rota de convite	`routes/convite.tsx`	Não existe; o convite é tratado pelos serviços e formulários de login/cadastro
Estrutura do backend	`backend/README.md` descreve `backend/src/…`	Os projetos estão diretamente em `backend/`
Índices do MongoDB	README cita índices em usuários, itens, e-mails, tokens…	Não encontrei criação explícita de índices em `MongoDbContext`

15.1 Pontos técnicos para evolução

- **Controllers acessando o MongoDB diretamente**: o próprio `backend/README.md` registra que `AuthController`, `CategoriasController`, `GroqController`, `RecuperarSenhaController` e `UsuarioController` consultam `MongoDbContext`. Mover essas consultas para serviços completaria a separação da Clean Architecture.
- **Helpers duplicados**: `PriceTextHelper` e `StoreAndBrandHelper` existem em `Application/Helpers` e em `Infrastructure/Helpers` com o mesmo tamanho; vale manter uma única cópia.
- **Controllers extensos**: `UsuarioController` (~870 linhas) e `RecuperarSenhaController` (~490 linhas) concentram muita lógica e seriam candidatos a refatoração em serviços menores.
- **Rate limiting em memória**: ver o ponto de atenção do capítulo 7.
- **Índices**: criar índices para `Itens` (`UsuarioId`, `CategoriaId`), `Usuarios` (`Email`, tokens) e `Categorias` (`UsuarioId`) melhora consultas conforme o volume cresce.
- **Cobertura de testes**: ampliar para controllers, `ResumoService` e lista pública.

16. Glossário

Termo	Significado
Enxoval	Conjunto de itens necessários para montar e equipar a casa nova
Cômodo	Ambiente da casa; na API corresponde a uma `Categoria`
VR/VA	Vale-refeição / vale-alimentação, tratado como forma de pagamento separada do dinheiro
Meta global	Orçamento total desejado para o enxoval
Origem	Como o item entra no plano (comprado, desejo, prometido, presente)
Slug	Trecho único da URL que identifica a lista pública de um casal
PWA	Progressive Web App: site instalável com Service Worker e funcionamento tipo aplicativo
VAPID	Padrão de identificação do servidor para envio de notificações Web Push
JWT	JSON Web Token, usado para assinar a sessão do usuário
Fallback	Alternativa usada automaticamente quando a opção principal falha
Optimistic update	Atualização da interface antes da confirmação do servidor, revertida em caso de erro
Clean Architecture	Organização em camadas em que o domínio não depende de detalhes técnicos